WikiArt Painting Classifier

Deep Learning Project Report

Miguel Venâncio[1] Gonçalo Torrão[1] Anastasiia Shulha[1] Leonor Ribeiro[1] Henrique Serrão[1]

[1]NOVA Information Management School (NOVA IMS), 2025/2026

github.com/vehnie/WikiArt-DeepLearning

Abstract

This report tackles artist attribution from painting images, framed as a 23-class classification task on a curated subset of WikiArt (13,340 images). We evaluate four architectures of increasing complexity: a shallow baseline CNN, a custom CNN with hyperparameter search, a fine-tuned ResNet50, and a Vision Transformer (ViT-B/16) trained with AdamW, cosine scheduling, and label smoothing. The ViT reaches **86.0% accuracy** and **0.850 F1-macro** on the held-out test set, outperforming the best CNN by 11.5 percentage points. A Grad-CAM analysis adapted for both CNN and Transformer architectures shows that the model focuses on stylistically meaningful regions, though it struggles with fine-grained distinctions between artists within the same movement. All code, trained models, and an interactive web application are publicly available.

1 Introduction

Artist attribution, the task of determining who painted a given work, requires recognising subtle stylistic signatures such as brushwork texture, colour palette preferences, compositional patterns, and subject matter. We formalise this as a supervised multi-class image classification problem.

Problem definition. Let $\mathcal{X} \subset \mathbb{R}^{H \times W \times 3}$ denote the space of RGB images resized to $H = W = 224$, and $\mathcal{Y} = \{0, 1, \ldots, C-1\}$ the label space with $C = 23$ artists. Given a training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with $N = 9{,}337$, we seek parameters θ^* that minimise the expected risk:

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\left[\mathcal{L}(f_\theta(\mathbf{x}), y)\right] \tag{1}$$

where $f_\theta : \mathcal{X} \to \Delta^{C-1}$ maps images to the probability simplex and $\mathcal{L}$ is the categorical cross-entropy. The predicted class is $\hat{y} = \arg\max_c \, f_\theta(\mathbf{x})_c$.

Challenges. Three properties of the dataset make this task difficult. First, there is *class imbalance*: a $3.9\times$ ratio separates the largest class (Vincent van Gogh, 1,322 images) from the smallest (Salvador Dalí, 336). Second, *inter-class similarity* is high, since Impressionist painters like Monet, Pissarro, and Hassam share colour palettes and compositional strategies. Third, *intra-class diversity* is significant, as prolific artists like Picasso span multiple stylistic periods. Because of the imbalance, we use **F1-macro** as the primary metric, which weights each class equally regardless of support.

Contributions. This work includes: (i) a systematic comparison of four architectures, from shallow CNNs to Vision Transformers; (ii) a Grad-CAM adaptation for ViT explainability that operates on patch token gradients; (iii) an interactive web application for real-time artist prediction; and (iv) a fully reproducible pipeline with code, trained models, and data splits publicly available.

2 Data Pipeline

Preprocessing. All images are resized to 224×224 pixels using Lanczos resampling and normalised to $[0, 1]$ via $\mathbf{x}_{\text{norm}} = \mathbf{x}_{\text{raw}}/255$. The dataset is partitioned via *stratified* random sampling into train/validation/test splits of 70%/15%/15% (9,337/2,001/2,002 images), preserving the per-class distribution across all partitions. Stratification is critical given the class imbalance.

Augmentation. Training images undergo stochastic augmentation via the `tf.data` pipeline: random horizontal flip ($p = 0.5$), brightness perturbation ($\delta \sim \mathcal{U}(-0.2, 0.2)$), contrast scaling ($\gamma \sim \mathcal{U}(0.8, 1.2)$), and saturation scaling ($s \sim \mathcal{U}(0.8, 1.2)$). These simulate natural variation in photographic reproductions without distorting artistic content. We deliberately exclude rotation, which could remove compositional cues relevant to artist style. Validation and test sets receive no augmentation.

Pipeline efficiency. Datasets are constructed as `tf.data.Dataset` objects with `AUTOTUNE` prefetch, overlapping CPU preprocessing with GPU training. Augmentation is applied lazily (on-the-fly) to avoid materialising augmented copies.

3 Model Architectures

We progressively explore four architectures, each addressing specific limitations of its predecessor. All models output a softmax distribution over $C = 23$ classes, trained with categorical cross-entropy and evaluated with accuracy and F1-macro.

3.1 Baseline CNN

A minimal 2-block convolutional network establishing the performance floor:

$$\mathbf{x} \xrightarrow{\text{Conv}(32,3\times3)} \xrightarrow{\text{MaxPool}} \xrightarrow{\text{Conv}(64,3\times3)} \xrightarrow{\text{MaxPool}} \xrightarrow{\text{Flatten}} \xrightarrow{\text{Dense}(128)} \xrightarrow{\text{Dropout}(0.5)} \xrightarrow{\text{Dense}(C)} \tag{2}$$

The Flatten operation produces a $54 \times 54 \times 64 = 186{,}624$-dimensional vector, resulting in $\sim$6.8M parameters dominated by the first fully-connected layer. Optimised with Adam ($\eta = 10^{-3}$), EarlyStopping (patience$= 5$, monitoring `val_loss`).

Limitations. The shallow receptive field ($\sim 10 \times 10$ pixels after two 3×3 convolutions and pooling) captures only local textures. The Flatten layer creates excessive parameters while discarding spatial structure. Training for 13 epochs yielded severe overfitting: 70.9% train accuracy vs. 36.1% test accuracy (34.8pp gap).

3.2 Custom CNN

A deeper architecture with architectural regularisation addressing the baseline's limitations:

$$\mathbf{x} \xrightarrow[\times 4]{\text{Conv}(k,3\times3)\to\text{BN}\to\text{ReLU}\to\text{MaxPool}} \xrightarrow{\text{GAP}} \xrightarrow{\text{Dense}(256)} \xrightarrow{\text{Dropout}(p)} \xrightarrow{\text{Dense}(C)} \tag{3}$$

where $k \in \{n, 2n, 4n, 8n\}$ with base filters n, and GAP denotes Global Average Pooling. Key improvements: (1) *Batch Normalisation* [6] after each convolution stabilises training by normalising activations; (2) GAP replaces Flatten, reducing the classifier input from $186{,}624$ to $8n$ dimensions; (3) four conv blocks increase the effective receptive field to capture multi-scale features.

Hyperparameter optimisation. We use Keras Tuner [7] with random search (50 trials, 10

epochs each) over: learning rate $\eta \in [10^{-4}, 10^{-2}]$ (log-uniform), dropout $p \in \{0.2, 0.3, 0.4, 0.5\}$, and base filters $n \in \{16, 32, 64\}$. The best configuration ($\eta = 4.24 \times 10^{-4}$, $p = 0.3$, $n = 32$) yields roughly 1.2M parameters, which is 5.7$\times$ fewer than the baseline, yet outperforms it by 18.8pp on test accuracy.

3.3 Transfer Learning with ResNet50

Instead of learning features from scratch, we use a ResNet50 [2] backbone pretrained on ImageNet (roughly 23.6M backbone parameters, 25.6M total with the classification head) and adapt it through progressive fine-tuning.

Architecture. Input images are rescaled from $[0, 1]$ to $[0, 255]$ and passed through the ResNet50 preprocessing function (channel-wise mean subtraction). The backbone feeds into a classification head: GAP, Dense(256, ReLU), Dropout(p), and Dense(C, softmax). During fine-tuning, the top K layers are unfrozen while BatchNorm layers remain frozen to preserve the running statistics estimated on ImageNet.

Systematic experiments. We conduct five experiments varying three factors:

Model	K	η	p	Val Acc	Observation
1	5	10^{-3}	0.5	0.552	Overfitting
2	10	10^{-3}	0.5	0.538	Overfitting
3	10	10^{-5}	0.6	**0.723**	Best generalisation
4	5	10^{-5}	0.4	0.688	Slight underfitting
5	10	10^{-5}	0.5	0.712	Competitive

Models 1–2 with $\eta = 10^{-3}$ show significant overfitting. Reducing to $\eta = 10^{-5}$ for Models 3–5 preserves ImageNet representations while allowing domain adaptation. Model 3 achieves the best validation performance with stronger dropout ($p = 0.6$) preventing overfitting despite 10 unfrozen layers. ReduceLROnPlateau (factor $= 0.5$, patience $= 2$) provides additional learning rate refinement. The best-performing configuration (Model 3, which achieved the highest test accuracy of 74.5%) was saved for final evaluation.

3.4 Vision Transformer (ViT-B/16)

The Vision Transformer [1] takes a fundamentally different approach from CNNs. Rather than applying hierarchical convolutions, it partitions the input image into a sequence of non-overlapping patches, linearly projects each one, and processes the resulting tokens through multi-head self-attention. This gives every layer a global receptive field from the start.

Patch embedding. An image $\mathbf{x} \in \mathbb{R}^{224\times224\times3}$ is split into $N = (224/16)^2 = 196$ patches of size $16 \times 16 \times 3$. Each patch is linearly embedded to dimension $d = 768$, prepended with a learnable `[CLS]` token, and summed with learnable positional embeddings:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{cls}}\ ;\ \mathbf{E}\mathbf{p}_1\ ;\ \ldots\ ;\ \mathbf{E}\mathbf{p}_N] + \mathbf{E}_{\text{pos}}, \quad \mathbf{E} \in \mathbb{R}^{d\times(16^2\cdot3)}, \quad \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times d} \tag{4}$$

Transformer encoder. The sequence passes through $L = 12$ encoder blocks, each applying multi-head self-attention (12 heads, $d_k = 64$) followed by an MLP with GELU activation:

$$\mathbf{z}'_l = \text{MSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1}, \qquad \mathbf{z}_l = \text{MLP}(\text{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l \tag{5}$$

where LN denotes Layer Normalisation and MSA is multi-head self-attention. The `[CLS]` token output $\mathbf{z}_L^0 \in \mathbb{R}^{768}$ serves as the image representation.

Classification head. On top of the backbone we place: Dropout(0.3), Dense(256, GELU), Dropout(0.3), and Dense(C, softmax). The intermediate 256-dimensional layer acts as a learned bottleneck, giving the classifier a richer representation than a direct linear projection from the 768-dim CLS token.

Training recipe. Fine-tuning large pretrained models on small datasets ($\sim$9K images for 85.8M parameters) requires careful regularisation. We apply four techniques, following established best practices for ViT fine-tuning:

(i) AdamW [4] with decoupled weight decay $\lambda = 0.01$. Unlike L2 regularisation folded into Adam's gradient updates, AdamW applies weight decay directly to the parameters: $\theta_{t+1} = (1 - \lambda\eta)\theta_t - \eta\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$. This decoupling leads to more uniform regularisation across parameters whose gradient magnitudes vary widely.

(ii) Cosine learning rate schedule with warmup. The learning rate follows:

$$\eta_t = \begin{cases} \eta_{\text{peak}} \cdot t/T_w & t < T_w \\ \dfrac{\eta_{\text{peak}}}{2}\left(1 + \cos\left(\pi \cdot \dfrac{t - T_w}{T - T_w}\right)\right) & t \geq T_w \end{cases} \tag{6}$$

with $\eta_{\text{peak}} = 3 \times 10^{-5}$, warmup steps $T_w = 0.1T$, and total steps $T = 11{,}680$ (20 epochs $\times$ 584 steps/epoch). The warmup phase prevents destabilising the pretrained weights with large early gradients, while cosine decay provides smooth convergence.

(iii) Label smoothing [5] with $\epsilon = 0.1$. The hard one-hot target $\mathbf{y}_{\text{hard}}$ is replaced by $\mathbf{y}_{\text{smooth}} = (1 - \epsilon)\mathbf{y}_{\text{hard}} + \epsilon/C$, discouraging the model from assigning probability mass too sharply and improving generalisation.

(iv) Dropout at $p = 0.3$ in both classification head layers, providing additional regularisation given the high parameter-to-sample ratio ($\sim$9K:85.8M).

4 Evaluation

All models are evaluated on the held-out test set (2,002 images, unseen during training and hyperparameter selection).

4.1 Overall Results

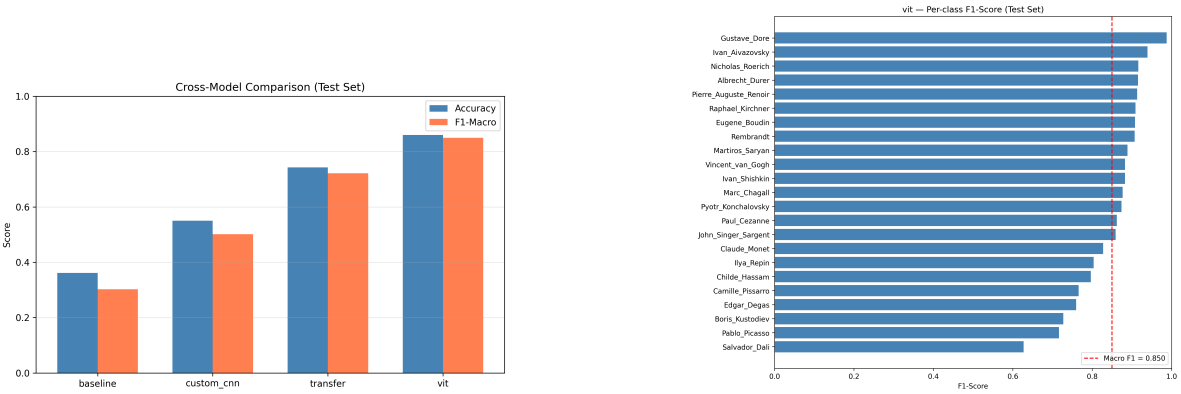
Table 1: Test set performance. All models trained with EarlyStopping on `val_loss`. Best results in bold.

Model	Accuracy	F1-Macro	Params	Epochs	Train Acc	Gap
Baseline CNN	36.1%	0.301	6.8M	13	70.9%	34.8pp
Custom CNN	54.9%	0.500	1.2M	17	67.1%	12.2pp
ResNet50 Transfer	74.5%	0.722	25.6M	9	95.1%	20.6pp
ViT-B/16	**86.0%**	**0.850**	85.8M	10	99.9%	13.9pp

Analysis. The results follow a clear progression. The Baseline CNN at 36.1% is well above random chance ($1/23 \approx 4.3$%) but shows that two convolutional layers are far from sufficient. The Custom CNN reaches 54.9% (+18.8pp), confirming that additional depth, BatchNorm, and hyperparameter tuning pay off. Transfer learning with ResNet50 pushes this to 74.5% (+19.6pp), which shows that low- and mid-level features learned from natu-

ral images (edges, textures, colour distributions) transfer well to paintings. Finally, the ViT reaches 86.0% (+11.5pp over ResNet50). We attribute this gap to the global self-attention mechanism, which captures compositional relationships across the full painting from the very first layer.

4.2 Per-Class Analysis



(a) Cross-model comparison. (b) ViT per-class F1 (macro = 0.850).

Figure 1: (a) Accuracy and F1-macro across all architectures. (b) Per-artist F1 for the ViT, ranging from Gustave Doré (0.98) to Salvador Dalí (0.62).

Looking at the per-class F1 scores (Figure 1b), a clear pattern emerges. Artists with highly *distinctive* visual signatures score above 0.90: Gustave Doré's engraving style, Ivan Aivazovsky's seascapes, and Nicholas Roerich's vivid mountain landscapes are all easy for the model to recognise. In contrast, artists who belong to *shared movements* score notably lower. Salvador Dalí (0.62) and Pablo Picasso (0.66), both modernists with overlapping abstract tendencies, are the hardest to separate.

4.3 Error Analysis

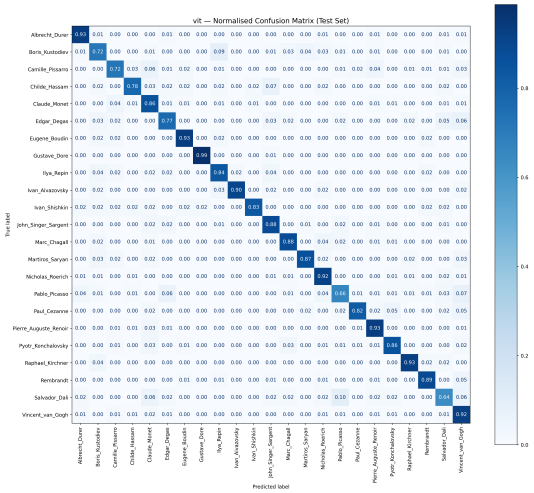


Figure 2: Normalised confusion matrix for ViT-B/16. Strong diagonal with off-diagonal clusters corresponding to stylistically similar artists within the same movement.

The confusion matrix (Figure 2) reveals systematic error patterns. The three most confused artist pairs are:

(i) Claude Monet ↔ Camille Pissarro. Both are core French Impressionists who painted similar plein-air landscapes with diffused light and broken colour. Their shared visual vocabulary makes this confusion understandable; even art historians sometimes disagree on attribution for certain works from this period.

(ii) Salvador Dalí ↔ Pablo Picasso. As 20th-century modernists, both experimented with abstract figurative forms and bold colour contrasts. The confusion tracks their shared Surrealist and Cubist influences.

(iii) Childe Hassam → Claude Monet. American Impressionism derives directly from the French school, so it is not surprising that Hassam is sometimes misidentified as Monet but not the reverse. This asymmetry is consistent with Hassam's acknowledged artistic debt to Monet.

5 Explainability via Grad-CAM

To interpret *what* the model attends to, we apply Gradient-weighted Class Activation Mapping (Grad-CAM) [3].

Standard Grad-CAM (CNNs). For a target class c, the gradient of the class score S^c with respect to the feature maps A^k of the last convolutional layer yields importance weights $\alpha_k^c = \frac{1}{Z}\sum_i \sum_j \frac{\partial S^c}{\partial A_{ij}^k}$. The heatmap is $L^c = \text{ReLU}(\sum_k \alpha_k^c A^k)$, highlighting regions that positively influence the predicted class.

Adapted Grad-CAM for ViT. Since ViT has no convolutional feature maps, standard Grad-CAM cannot be applied directly. Our adaptation computes gradients with respect to the **patch token outputs** of the backbone, excluding the `[CLS]` token. Let $\mathbf{z}_L^{1:N} \in \mathbb{R}^{N\times d}$ denote the final-layer patch representations. We compute:

$$w_j = \frac{1}{d}\sum_{k=1}^{d} \frac{\partial S^c}{\partial z_{L,j}^k}, \qquad H_{ij} = \text{ReLU}\left(\sum_k w_j \cdot z_{L,j}^k\right) \quad \text{reshaped to } 14\times 14 \tag{7}$$

This produces a spatial heatmap by treating each patch token's gradient-weighted activation as the importance of its corresponding 16×16 image region, then upsampling to the original resolution via bilinear interpolation.

Figure 3: Grad-CAM heatmaps for ViT-B/16. Top: original paintings. Bottom: heatmap overlay highlighting decisive regions. Green titles indicate correct predictions.

Findings. We generated 91 heatmaps (46 correct, 45 misclassified) covering all 23 artists. Three observations stand out. First, for correctly classified paintings the model attends to stylistically distinctive regions: engraving line-work for Doré, cross-hatching for Dürer, and broad colour fields for Van Gogh. Second, when the model fails it tends to fixate on features *shared* between the confused artists rather than discriminative ones. Third, a cross-class

comparison between Monet paintings (misclassified as Pissarro) and correctly classified Pissarro works shows nearly identical activation patterns, which suggests the model has not learned a strong boundary between these two Impressionists.

6 Discussion

Architecture tradeoffs. Performance correlates with the effective receptive field: from roughly 10×10 pixels in the baseline to the full 224×224 image in the ViT's first layer. For painting classification this matters because the *arrangement* of elements, overall colour harmony, and distribution of brushwork across the canvas carry as much information as local texture. Self-attention over 196 patches captures these relationships directly, whereas CNNs must build them up hierarchically through many layers of depth.

Regularisation matters. The ViT's strong performance (0.850 F1-macro) despite a high train–test gap (13.9pp) underscores the importance of the training recipe. AdamW, cosine scheduling, and label smoothing collectively prevent the 85.8M-parameter model from collapsing into overfitting on only 9K training images. For fine-tuning large pretrained models on small datasets, these regularisation choices are as consequential as the architecture itself.

The movement boundary problem. Our error analysis shows that confusion correlates strongly with art-historical proximity. Within Impressionism, for instance, artists deliberately shared techniques, so per-artist attribution is inherently ambiguous for certain works. This is less a model failure than a boundary problem in the dataset itself. Possible mitigations include hierarchical classification (first predict the movement, then the artist), exploiting non-stylistic cues such as signature regions or canvas texture, or contrastive learning with hard negative mining among artists within the same movement.

Limitations. Several caveats apply. The 23-artist subset may not generalise to the full WikiArt catalogue of 129+ artists. Photographic reproduction quality varies across the dataset, introducing noise unrelated to artistic style. The ViT's 85.8M parameters make it computationally expensive to deploy. Finally, some artists span multiple periods (Picasso's Blue, Rose, Cubist, and Surrealist phases, for example), and a single class label cannot capture this.

7 Conclusion

We built and evaluated a deep learning pipeline for artist classification on WikiArt paintings. The best model, a fine-tuned ViT-B/16 trained with AdamW, cosine scheduling, and label smoothing, reaches **86.0% accuracy** and **0.850 F1-macro** across 23 artists. The results show that global self-attention over image patches captures compositional relationships that convolutional models miss. Grad-CAM analysis confirms that the model picks up on artistically meaningful features, although stylistic overlap within movements remains the main source of error. The full codebase, trained models, and an interactive web application are available at https://github.com/vehnie/WikiArt-DeepLearning.

References

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021.

[2] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *CVPR*, pp. 770–778, 2016.

[3] R. R. Selvaraju, M. Cogswell, A. Das, et al. Grad-CAM: Visual explanations from deep networks via gradient-based localization. *ICCV*, pp. 618–626, 2017.

[4] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

[5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the Inception architecture for computer vision. *CVPR*, pp. 2818–2826, 2016.

[6] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*, pp. 448–456, 2015.

[7] T. O'Malley, E. Bursztein, J. Long, et al. KerasTuner. https://github.com/keras-team/keras-tuner, 2019.